

Project Work in Optimization Methods/Optimization Techniques for Machine Learning

Lecturer: **Matteo Lapucci**

`matteo.lapucci@unifi.it`

April 27, 2026

1 Project Work general rules

1. Students (both from INM and IAM programs) interested in carrying out the 3CFU project on Optimization can contact (anytime) me to discuss possible assignments. It is not required that the student(s) have already passed the exam of the main Optimization course, however basic knowledge of nonlinear optimization and some in depth study of a subset of the course topics will clearly be necessary for the project work.
2. In general, the project work will consist of the implementation of some optimization algorithm and possibly the (re)production of some numerical results on given problem instances, or the conduction of a small experimentation.
3. The project work is 3CFU worth, i.e., 75 hours of work per person on average.
4. The topic and the goals of the project work are established by the professor, together with the student(s), at the time of project assignment.
5. There is no deadline, however completion time will be one of the metrics of evaluation; students are encouraged to ask for a project work to be assigned when they are actually willing to work on it.
6. Project works can be either carried out individually or in groups of two students; groups of three students can exceptionally be accepted only if there are substantial reasons to do so. Of course, the workload grows proportionally.
7. There is no numerical grade associated with this module; evaluation is based on a quality assessment. The assessment is mainly based on 1) correctness of the work, 2) independence in carrying out the job, 3) awareness of the topic and critical spirit towards the results and 4) time required to complete the assignment.
8. To conclude the assignment, a short report shall be sent via email to the professor. The report should summarize all the work done, including mathematical formulations of problems and algorithms, relevant background theoretical results (if any), experiments description and results discussion. The report **MUST** be prepared with LaTeX and sent in pdf format.
9. After the approval of the report, there will be a final, brief oral discussion of the entire project work. Slides for presentation are not required, but feel free to prepare them if you find them useful for the discussion.

2 Assignment

Student(s) Francesco Marchini

Master's degree program Artificial Intelligence

Project topic SGD, with a properly tuned stepsize, was shown to be able to train many neural networks architecture at least as well as Adam. However, this seems not to be the case with language model tasks, where the use of optimizers like Adam appears to be necessary to successfully complete training.

A recent study [1] hypothesized and showed that this gap might be due to the heavy unbalance appearing among classes in this kind of problems. However, some further experiments hinted that the large gap in performance might be due to the combination of this particular setup and minibatching strategies.

To further deepen this analysis and isolate the complexities of this setting, experiments could be carried out in convex, heavily unbalanced two-classes problems (i.e., logistic regression with unbalanced data), analyzing both the full batch and mini-batch regime.

Project goals The student is asked to

- Implement (in Python language, exploit `numpy` library) classes and functions allowing to load data to build an instance of logistic regression problem, compute the loss and the gradient of the loss¹ (see Lecture notes, Sec. 6.3).
- Implement the following algorithms²:
 - Gradient descent with Armijo Line Search (FB);
 - Conjugate Gradient Method with Armijo line search and restart based on gradient-related conditions [2, Sec. 4.4.2] (FB);
 - Stochastic gradient descent with Polyak stepsize [3] (MB);
 - Adagrad [2, Sec. 8.1.2] (FB and MB);
 - RMSProp [2, Sec. 8.1.2] (FB and MB);
 - AdaDelta [2, Sec. 8.1.2] (FB and MB);
 - Adam [2, Sec. 8.1.2] (FB and MB);
- Generate a set of binary classification problems as follows:
 - exploit `make_classification`³ module from `sklearn`;
 - number of data points: $N \in \{500, 2000, 5000\}$;
 - number of features: $n = 16$;
 - fraction of positive samples $p \in \{0.1, 0.01, 0.005\}$.
- Test the effectiveness of the considered methods both in the full batch and minibatch settings.
 - Measure how the training loss decreases as the number of epochs goes by, plotting for each method the trend of the full loss and also the trend of the separated losses w.r.t. the two classes.
 - Additionally, report the model's accuracy on the two classes at the obtained final solutions.
 - As a reference to check correctness of implementation, the L-BFGS method can be applied (in a full-batch fashion) to retrieve the global optimum value (use L-BFGS-B that can be found in the `scipy.optimize` package).

¹tip: avoid nested for statements, prefer numpy builtin functions for working with tensor structures

²FB: Full Batch, MB: MiniBatch

³https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_classification.html

References

- [1] F. Kunstner et al., “Heavy-tailed class imbalance and why Adam outperforms gradient descent on language models.” *Advances in Neural Information Processing Systems* (2024)-
- [2] M. Lapucci (2025). Course Lecture Notes.
- [3] N. Loizou et al. “Stochastic Polyak step-size for SGD: An adaptive learning rate for fast convergence.” *International conference on artificial intelligence and statistics PMLR* (2021).